

Adaptive Resource Management Engine (ARME) with Context-Aware Prioritization

1. Core Architecture: Dynamic Priority Management System

A. Resource-Aware Task Manager

python

```
class AdaptiveResourceManager:
    def __init__(self, hardware_monitor, context_engine):
        self.hardware_monitor = hardware_monitor
        self.context_engine = context_engine
        self.task_registry = {}
        self.running_tasks = {}
        self.deferred_tasks = Queue()
        self.reflection_queue = Queue()
        self.resource_pools = {
            'critical': {'cpu': 0.3, 'memory': 0.4, 'npu': 0.2},
            'essential': {'cpu': 0.4, 'memory': 0.4, 'npu': 0.3},
            'background': {'cpu': 0.2, 'memory': 0.1, 'npu': 0.1},
            'reflection': {'cpu': 0.1, 'memory': 0.1, 'npu': 0.0}
        }

    def register_task(self, task_id, task_config):
        """
        Register a task with its resource profile and priority rules
        """
        self.task_registry[task_id] = {
            'function': task_config['function'],
            'base_priority': task_config.get('base_priority', 50),
            'resource_profile': self.calculate_resource_profile(task_config),
            'context_sensitivity': task_config.get('context_sensitivity', {}),
            'deferrable': task_config.get('deferrable', True),
            'max_deferral_time': task_config.get('max_deferral_time', 300),
            'reflection_analysis': task_config.get('reflection_analysis', True),
            'essential_override': task_config.get('essential_override', []),
            'performance_metrics': {
                'avg_execution_time': 0,
                'avg_cpu_usage': 0,
                'avg_memory_usage': 0,
                'success_rate': 1.0
            }
        }

    def calculate_dynamic_priority(self, task_id, current_context):
        """
        Calculate real-time priority based on context and system state
        """
        task = self.task_registry[task_id]

        # Base priority
        priority = task['base_priority']

        # Context adjustment
        context_boost = self.calculate_context_boost(task_id, current_context)
        priority += context_boost * 20
```

```

# Resource availability adjustment
resource_factor = self.calculate_resource_factor(task_id)
priority *= resource_factor

# Urgency factor (how long deferred)
if task_id in self.deferred_tasks:
    urgency = self.calculate_urgency(task_id)
    priority += urgency * 10

# Safety override (essential functions always high)
if self.is_essential_in_context(task_id, current_context):
    priority = min(priority, 90) # Cap at 90 for essential

return max(1, min(100, priority))

def calculate_context_boost(self, task_id, current_context):
    """
    Determine how important this task is in current context
    """
    task = self.task_registry[task_id]
    sensitivity = task['context_sensitivity']

    # Check for direct context match
    if current_context['primary'] in sensitivity:
        return sensitivity[current_context['primary']]

    # Check for secondary context relevance
    for secondary_context in current_context.get('secondary', []):
        if secondary_context in sensitivity:
            return sensitivity[secondary_context] * 0.7

    # Default low relevance
    return 0.1

def is_essential_in_context(self, task_id, current_context):
    """
    Check if task is essential in current context (cannot be deferred)
    """
    task = self.task_registry[task_id]
    essential_contexts = task.get('essential_override', [])

    # Always essential contexts
    always_essential = ['crisis', 'emergency', 'safety_breach']

    return (current_context['primary'] in always_essential or
            current_context['primary'] in essential_contexts)

```

B. Context-Aware Task Scheduler

python

```

class ContextAwareScheduler:
    def __init__(self, resource_manager):
        self.resource_manager = resource_manager
        self.context_history = deque(maxlen=100)
        self.context_transition_times = {}
        self.active_context = {
            'primary': 'normal',
            'secondary': [],
            'stability_score': 1.0,
            'duration': 0
        }

```

```

def update_context(self, sensor_data, user_interaction, system_state):
    """
    Determine current context based on multiple inputs
    """
    new_context = {
        'primary': self.determine_primary_context(sensor_data,
user_interaction),
        'secondary': self.determine_secondary_contexts(sensor_data,
user_interaction),
        'stability_score': self.calculate_context_stability(),
        'duration': self.calculate_context_duration(),
        'transition_likelihood': self.calculate_transition_likelihood()
    }

    # Log context transition
    if new_context['primary'] != self.active_context['primary']:
        self.log_context_transition(self.active_context, new_context)

    self.active_context = new_context
    self.context_history.append(new_context)

    # Update task priorities based on new context
    self.adjust_task_priorities_for_context()

    return new_context

def determine_primary_context(self, sensor_data, user_interaction):
    """
    Determine the primary operational context
    """
    # Check for safety emergencies
    if self.detect_safety_emergency(sensor_data):
        return 'crisis_emergency'

    # Check for user-engaged activities
    if user_interaction.get('activity_type') == 'tea_party':
        return 'tea_party_active'
    elif user_interaction.get('activity_type') == 'drawing':
        return 'creative_session'
    elif user_interaction.get('activity_type') == 'game_playing':
        return 'game_session'

    # Check for therapeutic states
    if sensor_data.get('user_emotional_state') in ['distressed', 'anxious']:
        return 'therapeutic_support'

    # Check for learning opportunities
    if self.detect_learning_opportunity(user_interaction):
        return 'learning_opportunity'

    # Default based on time and patterns
    return self.determine_default_context()

def determine_secondary_contexts(self, sensor_data, user_interaction):
    """
    Determine secondary contexts that modify primary behavior
    """
    secondary_contexts = []

    # Add resource-aware contexts
    hardware_state = self.resource_manager.hardware_monitor.get_state()
    if hardware_state['cpu_temp'] > 70:
        secondary_contexts.append('thermal_throttling')
    if hardware_state['cpu_usage'] > 85:

```

```

        secondary_contexts.append('high_load')

    # Add user state contexts
    if sensor_data.get('user_attention_level', 0.5) < 0.3:
        secondary_contexts.append('low_attention')
    if sensor_data.get('user_engagement', 0.5) > 0.8:
        secondary_contexts.append('high_engagement')

    # Add environmental contexts
    if sensor_data.get('ambient_noise', 0) > 70: # dB
        secondary_contexts.append('noisy_environment')
    if sensor_data.get('ambient_light', 0) < 20: # lux
        secondary_contexts.append('low_light')

    return secondary_contexts

def adjust_task_priorities_for_context(self):
    """
    Adjust all task priorities based on current context
    """
    for task_id in self.resource_manager.task_registry:
        new_priority = self.resource_manager.calculate_dynamic_priority(
            task_id,
            self.active_context
        )

        # Update task's effective priority
        self.resource_manager.task_registry[task_id]['effective_priority'] =
new_priority

        # If task is running and should be suspended
        if task_id in self.resource_manager.running_tasks:
            if self.should_suspend_task(task_id):
                self.suspend_task_for_reflection(task_id)

```

2. Three-Layer Task Execution System

A. Task Execution Orchestrator

python

```

class ThreeLayerTaskOrchestrator:
    def __init__(self, resource_manager):
        self.resource_manager = resource_manager
        self.execution_layers = {
            'real_time': {
                'max_tasks': 3,
                'priority_threshold': 80,
                'tasks': {},
                'resources': {'cpu': 0.4, 'memory': 0.5}
            },
            'near_real_time': {
                'max_tasks': 5,
                'priority_threshold': 60,
                'tasks': {},
                'resources': {'cpu': 0.3, 'memory': 0.3}
            },
            'background': {
                'max_tasks': 10,
                'priority_threshold': 30,
                'tasks': {},

```

```

        'resources': {'cpu': 0.2, 'memory': 0.1}
    }
}

def execute_task(self, task_id, input_data=None):
    """
    Execute task in appropriate layer based on priority and resources
    """
    task = self.resource_manager.task_registry[task_id]
    priority = task.get('effective_priority', task['base_priority'])

    # Determine execution layer
    if priority >= self.execution_layers['real_time']['priority_threshold']:
        layer = 'real_time'
    elif priority >= self.execution_layers['near_real_time']
['priority_threshold']:
        layer = 'near_real_time'
    else:
        layer = 'background'

    # Check if layer has capacity
    if not self.layer_has_capacity(layer):
        # Try to move to higher layer if possible
        layer = self.find_available_layer(priority)
        if not layer:
            # No capacity - defer task
            return self.defer_task(task_id, input_data)

    # Execute in selected layer
    return self.execute_in_layer(task_id, layer, input_data)

def execute_in_layer(self, task_id, layer, input_data):
    """
    Execute task with layer-specific resource constraints
    """
    task_config = self.resource_manager.task_registry[task_id]

    # Create constrained execution environment
    constraints = self.execution_layers[layer]['resources']

    if layer == 'real_time':
        # Execute immediately with priority
        result = self.execute_realtime(task_config['function'], input_data,
constraints)

    elif layer == 'near_real_time':
        # Execute with small delay
        result = self.execute_near_realtime(task_config['function'],
input_data, constraints)

    else: # background
        # Execute when resources available
        result = self.execute_background(task_config['function'],
input_data, constraints)

    # Track execution
    self.track_execution_metrics(task_id, layer, result)

    return result

def execute_realtime(self, task_function, input_data, constraints):
    """
    Execute with real-time constraints
    """

```

```

        # Apply CPU limiting
        with CPULimiter(constraints['cpu']):
            # Apply memory limiting
            with MemoryLimiter(constraints['memory'] * 1024): # Convert to MB
                # Execute with timeout
                try:
                    result = task_function(input_data)
                    return {'status': 'success', 'result': result, 'layer':
'real_time'}
                except TimeoutError:
                    return {'status': 'timeout', 'layer': 'real_time',
'suggested': 'defer'}
                except Exception as e:
                    return {'status': 'error', 'error': str(e), 'layer':
'real_time'}

    def execute_background(self, task_function, input_data, constraints):
        """
        Execute in background with checkpointing
        """
        # Save input data to disk
        checkpoint_id = self.create_checkpoint(input_data)

        # Execute in low-priority thread
        def background_worker():
            try:
                # Load from checkpoint
                loaded_data = self.load_checkpoint(checkpoint_id)

                # Execute with aggressive limits
                with CPULimiter(constraints['cpu'] * 0.5): # Use half of
allocated
                    result = task_function(loaded_data)

                # Save result for later retrieval
                self.save_result(checkpoint_id, result)

                # Cleanup
                self.delete_checkpoint(checkpoint_id)

            except Exception as e:
                # Log error for reflection analysis
                self.log_background_error(checkpoint_id, str(e))

        # Start background thread
        thread = threading.Thread(target=background_worker, daemon=True)
        thread.start()

        return {
            'status': 'deferred',
            'checkpoint_id': checkpoint_id,
            'layer': 'background',
            'estimated_completion': self.estimate_completion_time(task_function)
        }

```

B. Intelligent Task Deferral System

python

```

class IntelligentDeferralSystem:
    def __init__(self, storage_path):
        self.storage_path = storage_path
        self.deferred_tasks = {}

```

```

self.deferral_reasons = {}
self.resumption_triggers = {}

def defer_task(self, task_id, input_data, reason, estimated_importance):
    """
    Defer task with metadata for later execution
    """
    deferral_id = f"{task_id}_{int(time.time())}"

    # Create deferral package
    deferral_package = {
        'task_id': task_id,
        'input_data': input_data,
        'timestamp': time.time(),
        'reason': reason,
        'estimated_importance': estimated_importance,
        'dependencies': self.get_task_dependencies(task_id),
        'context_at_deferral': self.get_current_context(),
        'expected_resource_requirements':
self.estimate_resource_requirements(task_id),
        'optimal_execution_conditions':
self.determine_optimal_conditions(task_id)
    }

    # Save to storage
    self.save_to_storage(deferral_id, deferral_package)

    # Add to active deferral tracking
    self.deferred_tasks[deferral_id] = {
        'status': 'waiting',
        'priority': estimated_importance,
        'next_check': time.time() + 60 # Check in 1 minute
    }

    # Set up resumption triggers
    self.setup_resumption_triggers(deferral_id, deferral_package)

    return deferral_id

def setup_resumption_triggers(self, deferral_id, deferral_package):
    """
    Set up conditions for when to resume this task
    """
    triggers = []

    # Resource availability trigger
    triggers.append({
        'type': 'resource_available',
        'conditions': {
            'cpu_available':
deferral_package['expected_resource_requirements']['cpu'],
            'memory_available':
deferral_package['expected_resource_requirements']['memory']
        },
        'check_interval': 30 # Check every 30 seconds
    })

    # Context match trigger
    if
deferral_package['optimal_execution_conditions'].get('preferred_contexts'):
        triggers.append({
            'type': 'context_match',
            'conditions': deferral_package['optimal_execution_conditions']
['preferred_contexts'],

```

```

        'check_interval': 10
    })

    # Time-based trigger (don't defer forever)
    triggers.append({
        'type': 'timeout',
        'max_wait': deferral_package.get('max_deferral_time', 300),
        'check_interval': 60
    })

    self.resumption_triggers[deferral_id] = triggers

def check_resumption_conditions(self):
    """
    Check if any deferred tasks can be resumed
    """
    resumable_tasks = []

    for deferral_id, task_info in self.deferred_tasks.items():
        if task_info['status'] == 'waiting' and time.time() >
task_info['next_check']:
            # Check all triggers
            can_resume = self.evaluate_resumption_triggers(deferral_id)

            if can_resume:
                resumable_tasks.append(deferral_id)

            # Update next check time
            task_info['next_check'] = time.time() +
self.calculate_next_check_interval(deferral_id)

    return resumable_tasks

def evaluate_resumption_triggers(self, deferral_id):
    """
    Evaluate all triggers for a deferred task
    """
    triggers = self.resumption_triggers.get(deferral_id, [])

    for trigger in triggers:
        if trigger['type'] == 'resource_available':
            if self.check_resource_availability(trigger['conditions']):
                return True

        elif trigger['type'] == 'context_match':
            if self.check_context_match(trigger['conditions']):
                return True

        elif trigger['type'] == 'timeout':
            # Load deferral package to check timestamp
            package = self.load_from_storage(deferral_id)
            if time.time() - package['timestamp'] > trigger['max_wait']:
                return True # Timeout reached, execute anyway

    return False

def resume_task(self, deferral_id):
    """
    Resume a deferred task
    """
    # Load deferral package
    package = self.load_from_storage(deferral_id)

    # Update status

```

```

self.deferred_tasks[deferral_id]['status'] = 'resuming'

# Execute task with recorded context
result = self.execute_with_context(package)

# Clean up
self.cleanup_deferral(deferral_id)

return result

```

3. Reflection Engine for Deep Analysis

A. Reflection Period Management

python

```

class ReflectionEngine:
    def __init__(self, storage_path, analysis_modules):
        self.storage_path = storage_path
        self.analysis_modules = analysis_modules
        self.reflection_queue = []
        self.analysis_results = {}
        self.learning_updates = {}

    def schedule_reflection(self, data_type, data, priority,
analysis_requirements):
        """
        Schedule data for reflection analysis
        """
        reflection_item = {
            'id': f"{data_type}_{int(time.time())}",
            'data_type': data_type,
            'data': data,
            'timestamp': time.time(),
            'priority': priority,
            'analysis_requirements': analysis_requirements,
            'status': 'pending',
            'scheduled_for': self.calculate_reflection_time(priority)
        }

        # Save to persistent storage
        self.save_reflection_item(reflection_item)

        # Add to queue
        self.reflection_queue.append(reflection_item['id'])

        # Sort queue by priority and scheduling time
        self.reflection_queue.sort(key=lambda x:
self.get_reflection_priority(x))

        return reflection_item['id']

    def calculate_reflection_time(self, priority):
        """
        Determine when reflection should occur based on priority
        """
        base_times = {
            'high': 300,      # 5 minutes
            'medium': 1800,   # 30 minutes
            'low': 3600       # 1 hour
        }

```

```

        base_time = base_times.get(priority, 1800)

        # Adjust based on system idle patterns
        idle_patterns = self.analyze_idle_patterns()
        optimal_time = self.find_optimal_reflection_window(base_time,
idle_patterns)

        return time.time() + optimal_time

def run_reflection_cycle(self):
    """
    Execute reflection analysis when system is idle
    """
    # Check if system is suitable for reflection
    if not self.is_suitable_for_reflection():
        return

    # Get next reflection item
    if not self.reflection_queue:
        return

    reflection_id = self.reflection_queue.pop(0)
    reflection_item = self.load_reflection_item(reflection_id)

    # Update status
    reflection_item['status'] = 'analyzing'
    self.save_reflection_item(reflection_item)

    # Perform analysis based on requirements
    analysis_results = {}
    for requirement in reflection_item['analysis_requirements']:
        if requirement in self.analysis_modules:
            result = self.analysis_modules[requirement].analyze(
                reflection_item['data']
            )
            analysis_results[requirement] = result

    # Generate insights
    insights = self.generate_insights(analysis_results)

    # Update learning
    learning_updates = self.update_learning_models(insights)

    # Store results
    reflection_item['status'] = 'completed'
    reflection_item['analysis_results'] = analysis_results
    reflection_item['insights'] = insights
    reflection_item['learning_updates'] = learning_updates
    reflection_item['completed_at'] = time.time()

    self.save_reflection_item(reflection_item)

    # Cleanup old reflections
    self.cleanup_old_reflections()

    return insights

def generate_insights(self, analysis_results):
    """
    Generate actionable insights from analysis results
    """
    insights = {
        'behavioral_patterns': [],

```

```

        'system_optimizations': [],
        'user_preferences': [],
        'context_understanding': [],
        'anomalies': []
    }

    # Behavioral analysis insights
    if 'behavioral_analysis' in analysis_results:
        behavior_data = analysis_results['behavioral_analysis']

        # Detect patterns
        patterns = self.detect_behavioral_patterns(behavior_data)
        insights['behavioral_patterns'].extend(patterns)

        # Detect anomalies
        anomalies = self.detect_behavioral_anomalies(behavior_data)
        insights['anomalies'].extend(anomalies)

    # System performance insights
    if 'performance_analysis' in analysis_results:
        perf_data = analysis_results['performance_analysis']

        # Optimization suggestions
        optimizations = self.generate_optimization_suggestions(perf_data)
        insights['system_optimizations'].extend(optimizations)

    # User interaction insights
    if 'interaction_analysis' in analysis_results:
        interaction_data = analysis_results['interaction_analysis']

        # Preference detection
        preferences = self.detect_user_preferences(interaction_data)
        insights['user_preferences'].extend(preferences)

        # Context understanding
        context_insights =
self.understand_context_patterns(interaction_data)
        insights['context_understanding'].extend(context_insights)

    return insights

def update_learning_models(self, insights):
    """
    Update system learning models based on reflection insights
    """
    updates = {}

    # Update behavioral models
    for pattern in insights.get('behavioral_patterns', []):
        if pattern['type'] == 'interaction_pattern':
            update = self.update_interaction_model(pattern)
            updates.setdefault('interaction_model', []).append(update)

    # Update resource management models
    for optimization in insights.get('system_optimizations', []):
        if optimization['type'] == 'resource_allocation':
            update = self.update_resource_model(optimization)
            updates.setdefault('resource_model', []).append(update)

    # Update context understanding
    for context_insight in insights.get('context_understanding', []):
        update = self.update_context_model(context_insight)
        updates.setdefault('context_model', []).append(update)

```

```
return updates
```

B. Analysis Modules for Reflection

python

```
class BehavioralAnalysisModule:
    def __init__(self):
        self.pattern_detectors = {
            'engagement_patterns': EngagementPatternDetector(),
            'emotional_patterns': EmotionalPatternDetector(),
            'interaction_sequences': InteractionSequenceAnalyzer(),
            'preference_evolution': PreferenceEvolutionTracker()
        }

    def analyze(self, behavioral_data):
        """
        Analyze behavioral data for patterns and insights
        """
        results = {}

        for detector_name, detector in self.pattern_detectors.items():
            try:
                result = detector.analyze(behavioral_data)
                results[detector_name] = result
            except Exception as e:
                logger.error(f"Behavioral analysis error in {detector_name}: {e}")

        # Synthesize comprehensive insights
        synthesized = self.synthesize_insights(results)
        results['synthesized_insights'] = synthesized

        return results

    def synthesize_insights(self, detector_results):
        """
        Synthesize insights from multiple detectors
        """
        insights = []

        # Cross-reference engagement and emotional patterns
        engagement_patterns = detector_results.get('engagement_patterns',
        {}).get('patterns', [])
        emotional_patterns = detector_results.get('emotional_patterns',
        {}).get('patterns', [])

        for eng_pattern in engagement_patterns:
            for emo_pattern in emotional_patterns:
                # Check for correlation
                if self.patterns_correlate(eng_pattern, emo_pattern):
                    insight = {
                        'type': 'engagement_emotion_correlation',
                        'description': f"When {eng_pattern['description']}, user
tends to feel {emo_pattern['description']}",
                        'confidence': min(eng_pattern['confidence'],
emo_pattern['confidence']),
                        'implications': self.derive_implications(eng_pattern,
emo_pattern)
                    }
                    insights.append(insight)

        # Analyze preference evolution
```

```

        preference_data = detector_results.get('preference_evolution', {})
        if preference_data.get('significant_changes'):
            for change in preference_data['significant_changes']:
                insight = {
                    'type': 'preference_evolution',
                    'description': f"User's preference for
{change['preference']} has {change['direction']}",
                    'magnitude': change['magnitude'],
                    'suggested_adjustments':
self.suggest_preference_adjustments(change)
                }
                insights.append(insight)

        return insights

class SystemPerformanceAnalysisModule:
    def __init__(self):
        self.performance_metrics = {
            'response_times': [],
            'resource_usage': [],
            'task_completion': [],
            'thermal_performance': []
        }

    def analyze(self, performance_data):
        """
        Analyze system performance data
        """
        analysis = {
            'bottlenecks': self.identify_bottlenecks(performance_data),
            'optimization_opportunities':
self.find_optimization_opportunities(performance_data),
            'thermal_management':
self.analyze_thermal_performance(performance_data),
            'resource_efficiency':
self.calculate_resource_efficiency(performance_data)
        }

        # Generate recommendations
        recommendations = self.generate_recommendations(analysis)
        analysis['recommendations'] = recommendations

        return analysis

    def identify_bottlenecks(self, performance_data):
        """
        Identify performance bottlenecks
        """
        bottlenecks = []

        # Analyze response times
        response_times = performance_data.get('response_times', [])
        if response_times:
            avg_response = sum(response_times) / len(response_times)
            if avg_response > 1000: # 1 second threshold
                bottlenecks.append({
                    'type': 'response_time',
                    'metric': 'average_response_time',
                    'value': avg_response,
                    'threshold': 1000,
                    'suggestion': 'Optimize task scheduling or reduce task
complexity'
                })

```

```

# Analyze CPU usage patterns
cpu_data = performance_data.get('cpu_usage', [])
if cpu_data:
    high_usage_periods = [usage for usage in cpu_data if usage > 80]
    if len(high_usage_periods) > len(cpu_data) * 0.3: # 30% of time
        bottlenecks.append({
            'type': 'cpu_contention',
            'metric': 'high_usage_percentage',
            'value': len(high_usage_periods) / len(cpu_data),
            'suggestion': 'Implement better task deferral or increase
background task delays'
        })

    return bottlenecks

def generate_recommendations(self, analysis):
    """
    Generate actionable recommendations
    """
    recommendations = []

    for bottleneck in analysis.get('bottlenecks', []):
        if bottleneck['type'] == 'response_time':
            recommendations.append({
                'action': 'adjust_task_priorities',
                'description': 'Increase priority for time-sensitive tasks',
                'expected_improvement': '20-30% reduction in average
response time',
                'implementation_difficulty': 'low'
            })

        elif bottleneck['type'] == 'cpu_contention':
            recommendations.append({
                'action': 'implement_adaptive_deferral',
                'description': 'Defer non-essential tasks during high CPU
usage',
                'expected_improvement': 'Reduced thermal throttling,
smoother performance',
                'implementation_difficulty': 'medium'
            })

    # Add recommendations from optimization opportunities
    for opportunity in analysis.get('optimization_opportunities', []):
        recommendations.append({
            'action': opportunity['optimization'],
            'description': opportunity['description'],
            'expected_improvement': opportunity['expected_benefit'],
            'implementation_difficulty': opportunity.get('difficulty',
'medium')
        })

    return recommendations

```

4. Context-Specific Resource Allocation

A. Activity-Based Resource Profiles

python

```
class ActivityResourceProfiles:
```

```

def __init__(self):
    self.profiles = {
        'tea_party_active': {
            'description': 'Interactive tea party with toys',
            'essential_tasks': [
                'toy_recognition',
                'child_emotion_detection',
                'marvin_expression_render',
                'ar_overlay_render',
                'voice_interaction'
            ],
            'background_tasks': [
                'behavioral_analysis_full',
                'conversation_analysis',
                'system_performance_logging',
                'long_term_memory_update'
            ],
            'resource_allocation': {
                'cpu': {'essential': 0.6, 'background': 0.2, 'reflection':
0.0},
                'memory': {'essential': 0.7, 'background': 0.2,
'reflection': 0.0},
                'npu': {'essential': 0.8, 'background': 0.1, 'reflection':
0.0}
            },
            'deferral_rules': {
                'allow_deferral': True,
                'max_deferral_time': 600, # 10 minutes
                'resumption_triggers': ['user_idle', 'activity_end']
            }
        },
        'creative_session': {
            'description': 'Drawing/coloring activity',
            'essential_tasks': [
                'drawing_canvas_render',
                'tool_interaction',
                'color_palette_management',
                'basic_ai_assistance'
            ],
            'background_tasks': [
                'drawing_pattern_analysis',
                'creative_style_learning',
                'progress_tracking',
                'therapeutic_benefit_analysis'
            ],
            'resource_allocation': {
                'cpu': {'essential': 0.5, 'background': 0.3, 'reflection':
0.1},
                'memory': {'essential': 0.6, 'background': 0.3,
'reflection': 0.0},
                'npu': {'essential': 0.4, 'background': 0.3, 'reflection':
0.1}
            }
        },
        'crisis_emergency': {
            'description': 'Safety emergency detected',
            'essential_tasks': [
                'safety_monitoring',
                'emergency_response',
                'alert_generation',
                'user_state_assessment'
            ],

```

```

        'background_tasks': [], # No background tasks during crisis
        'resource_allocation': {
            'cpu': {'essential': 0.9, 'background': 0.0, 'reflection':
0.0},
            'memory': {'essential': 0.9, 'background': 0.0,
'reflection': 0.0},
            'npu': {'essential': 0.8, 'background': 0.0, 'reflection':
0.0}
        },
        'deferral_rules': {
            'allow_deferral': False,
            'max_deferral_time': 0,
            'resumption_triggers': []
        }
    },

    'reflection_period': {
        'description': 'System idle, running reflection tasks',
        'essential_tasks': [
            'system_health_monitoring',
            'resource_cleanup'
        ],
        'background_tasks': [
            'behavioral_analysis_full',
            'performance_analysis',
            'learning_model_updates',
            'data_compression',
            'storage_optimization'
        ],
        'resource_allocation': {
            'cpu': {'essential': 0.2, 'background': 0.7, 'reflection':
0.1},
            'memory': {'essential': 0.3, 'background': 0.6,
'reflection': 0.1},
            'npu': {'essential': 0.1, 'background': 0.3, 'reflection':
0.4}
        }
    }
}

```

```

def get_profile(self, activity_type):
    """
    Get resource profile for specific activity
    """
    return self.profiles.get(activity_type, self.profiles['normal'])

def adjust_profile_for_conditions(self, profile, system_conditions):
    """
    Adjust profile based on current system conditions
    """
    adjusted_profile = profile.copy()

    # Reduce resource allocation if thermal throttling
    if system_conditions.get('thermal_throttling'):
        for resource in adjusted_profile['resource_allocation']:
            for category in adjusted_profile['resource_allocation']
[resource]:
                adjusted_profile['resource_allocation'][resource][category]
*= 0.7

    # Reduce background tasks if low memory
    if system_conditions.get('memory_pressure'):
        # Move some background tasks to reflection-only
        if len(adjusted_profile['background_tasks']) > 2:

```

```

        moved_tasks = adjusted_profile['background_tasks'][2:]
        adjusted_profile['background_tasks'] =
adjusted_profile['background_tasks'][:2]
        adjusted_profile.setdefault('reflection_only_tasks',
[]) .extend(moved_tasks)

    return adjusted_profile

```

B. Dynamic Resource Reallocation

python

```

class DynamicResourceReallocator:
    def __init__(self, resource_manager, activity_profiles):
        self.resource_manager = resource_manager
        self.activity_profiles = activity_profiles
        self.current_allocation = {}
        self.allocation_history = []

    def reallocate_for_activity(self, activity_type, urgency=0.5):
        """
        Reallocate resources for new activity
        """
        # Get profile for activity
        profile = self.activity_profiles.get_profile(activity_type)

        # Adjust based on urgency
        if urgency > 0.8: # High urgency
            profile = self.boost_essential_resources(profile, 1.2)

        # Apply allocation
        self.apply_allocation(profile['resource_allocation'])

        # Update task priorities
        self.update_task_priorities_for_profile(profile)

        # Log reallocation
        self.log_reallocation(activity_type, profile)

        return profile

    def apply_allocation(self, allocation):
        """
        Apply resource allocation to system
        """
        for resource_type, categories in allocation.items():
            for category, percentage in categories.items():
                # Convert percentage to absolute based on system capacity
                absolute = self.percentage_to_absolute(resource_type,
percentage)

                # Apply limits
                self.set_resource_limit(resource_type, category, absolute)

                # Update current allocation tracking
                self.current_allocation[f"{resource_type}_{category}"] =
absolute

        # Save to history
        self.allocation_history.append({
            'timestamp': time.time(),
            'allocation': allocation,
            'system_state': self.get_system_state()

```

```

    })

def update_task_priorities_for_profile(self, profile):
    """
    Update task priorities based on activity profile
    """
    # Boost essential tasks
    for task_id in profile['essential_tasks']:
        if task_id in self.resource_manager.task_registry:
            current_priority =
self.resource_manager.task_registry[task_id].get('effective_priority', 50)
            new_priority = min(95, current_priority + 20) # Boost but don't
max out
            self.resource_manager.task_registry[task_id]
['effective_priority'] = new_priority

    # Reduce background tasks
    for task_id in profile['background_tasks']:
        if task_id in self.resource_manager.task_registry:
            current_priority =
self.resource_manager.task_registry[task_id].get('effective_priority', 50)
            new_priority = max(10, current_priority - 15) # Reduce but keep
minimal
            self.resource_manager.task_registry[task_id]
['effective_priority'] = new_priority

    # Defer reflection-only tasks
    for task_id in profile.get('reflection_only_tasks', []):
        if task_id in self.resource_manager.task_registry:
            # Mark for deferral
            self.resource_manager.task_registry[task_id]
['defer_until_reflection'] = True

def monitor_and_adjust(self):
    """
    Continuously monitor and adjust resource allocation
    """
    # Check system conditions
    system_state = self.get_system_state()

    # Adjust for thermal conditions
    if system_state['cpu_temp'] > 75:
        self.reduce_allocation_for_thermal(0.7) # Reduce to 70%

    # Adjust for memory pressure
    if system_state['memory_used'] > 0.9: # 90% memory used
        self.reduce_background_allocation(0.5) # Cut background in half

    # Check for idle periods to run reflection
    if self.detect_idle_period():
        self.trigger_reflection_period()

def trigger_reflection_period(self):
    """
    Trigger reflection period when system is idle
    """
    # Save current state
    previous_state = {
        'allocation': self.current_allocation.copy(),
        'active_tasks': list(self.resource_manager.running_tasks.keys())
    }

    # Switch to reflection profile
    reflection_profile =

```

```

self.activity_profiles.get_profile('reflection_period')
    self.reallocate_for_activity('reflection_period')

    # Run reflection for limited time
    reflection_duration = self.calculate_optimal_reflection_duration()

    # Start reflection timer
    self.start_reflection_timer(reflection_duration, previous_state)

def start_reflection_timer(self, duration, previous_state):
    """
    Start timer to return to previous state after reflection
    """
    def return_to_previous():
        # Restore previous allocation
        self.apply_allocation(previous_state['allocation'])

        # Restore task priorities
        self.restore_task_priorities(previous_state['active_tasks'])

        # Process any reflection results
        self.process_reflection_results()

    # Schedule return
    threading.Timer(duration, return_to_previous).start()

```

5. Implementation Strategy

A. Integration with Existing System

python

```

class ARMEIntegration:
    def __init__(self, marvin_core):
        self.marvin_core = marvin_core
        self.resource_manager = AdaptiveResourceManager(
            marvin_core.hardware_monitor,
            marvin_core.context_engine
        )
        self.scheduler = ContextAwareScheduler(self.resource_manager)
        self.orchestrator = ThreeLayerTaskOrchestrator(self.resource_manager)
        self.deferral_system = IntelligentDeferralSystem('deferred_tasks')
        self.reflection_engine = ReflectionEngine('reflection_data', {
            'behavioral': BehavioralAnalysisModule(),
            'performance': SystemPerformanceAnalysisModule(),
            'interaction': InteractionAnalysisModule()
        })

        # Register all existing tasks
        self.register_all_tasks()

    def register_all_tasks(self):
        """
        Register all system tasks with resource profiles
        """
        tasks = {
            # Tea party tasks
            'toy_recognition': {
                'function': self.marvin_core.toy_recognition.recognize_toys,
                'base_priority': 85,
                'context_sensitivity': {

```

```

        'tea_party_active': 1.0,
        'normal': 0.3,
        'reflection_period': 0.1
    },
    'deferrable': True,
    'max_deferral_time': 300,
    'reflection_analysis': True,
    'essential_override': ['crisis_emergency']
},

'child_emotion_detection': {
    'function': self.marvin_core.emotion_recognizer.analyze,
    'base_priority': 90,
    'context_sensitivity': {
        'tea_party_active': 1.0,
        'creative_session': 0.8,
        'crisis_emergency': 1.0,
        'normal': 0.5
    },
    'deferrable': False, # Never defer emotion detection
    'reflection_analysis': True
},

# Drawing program tasks
'drawing_canvas_render': {
    'function': self.marvin_core.drawing_engine.render,
    'base_priority': 80,
    'context_sensitivity': {
        'creative_session': 1.0,
        'normal': 0.2
    },
    'deferrable': True,
    'max_deferral_time': 600
},

# Safety tasks
'safety_monitoring': {
    'function': self.marvin_core.safety_monitor.check,
    'base_priority': 95, # Always high
    'context_sensitivity': {
        'crisis_emergency': 1.0,
        'tea_party_active': 0.9,
        'creative_session': 0.9,
        'normal': 0.8,
        'reflection_period': 0.8
    },
    'deferrable': False, # Never defer safety
    'essential_override': ['all'] # Essential in all contexts
}
}

for task_id, config in tasks.items():
    self.resource_manager.register_task(task_id, config)

def execute_with_priority(self, task_id, input_data=None):
    """
    Execute task with priority management
    """
    # Check if task should be deferred
    current_context = self.scheduler.active_context

    if self.should_defer_task(task_id, current_context):
        # Defer with appropriate priority
        deferral_priority = self.calculate_deferral_priority(task_id,

```

```

current_context)
    deferral_id = self.deferral_system.defer_task(
        task_id,
        input_data,
        reason='context_not_optimal',
        estimated_importance=deferral_priority
    )
    return {'status': 'deferred', 'deferral_id': deferral_id}

# Execute with appropriate resource allocation
return self.orchestrator.execute_task(task_id, input_data)

def run_system_cycle(self):
    """
    Main system cycle with adaptive resource management
    """
    # 1. Update context
    context = self.scheduler.update_context(
        self.marvin_core.get_sensor_data(),
        self.marvin_core.get_user_interaction(),
        self.marvin_core.get_system_state()
    )

    # 2. Check for deferred tasks that can resume
    resumable = self.deferral_system.check_resumption_conditions()
    for deferral_id in resumable:
        self.deferral_system.resume_task(deferral_id)

    # 3. Check if reflection should run
    if self.should_run_reflection():
        insights = self.reflection_engine.run_reflection_cycle()
        if insights:
            self.apply_reflection_insights(insights)

    # 4. Monitor and adjust resource allocation
    self.resource_manager.monitor_and_adjust()

    # 5. Log current state for future reflection
    self.log_system_state_for_reflection(context)

```

B. Configuration Files

resource_profiles.json

json

```

{
  "task_profiles": {
    "tea_party_active": {
      "essential_tasks": ["toy_recognition", "child_emotion_detection",
"marvin_expression_render"],
      "background_tasks": ["behavioral_analysis", "conversation_logging"],
      "resource_limits": {
        "cpu": {"essential": 0.6, "background": 0.2},
        "memory": {"essential": 500, "background": 100},
        "thermal_limit": 75
      }
    },
    "reflection_period": {
      "essential_tasks": ["system_health"],
      "background_tasks": ["behavioral_analysis_full",
"performance_optimization"],
      "resource_limits": {

```

```

        "cpu": {"essential": 0.2, "background": 0.5},
        "memory": {"essential": 200, "background": 300},
        "thermal_limit": 70
    }
}
},
"deferral_rules": {
    "max_deferral_time_default": 300,
    "resumption_triggers": {
        "resource_available": {"cpu": 0.4, "memory": 0.5},
        "context_match": {"similarity_threshold": 0.7},
        "time_based": {"min_interval": 60}
    }
},
"reflection_schedule": {
    "idle_threshold": 60,
    "minimum_duration": 30,
    "maximum_duration": 300,
    "priority_mapping": {
        "high": 300,
        "medium": 1800,
        "low": 3600
    }
}
}
}

```

task_priorities.json

json

```

{
    "priority_levels": {
        "critical": {
            "range": [90, 100],
            "description": "Safety and essential functions",
            "deferrable": false,
            "examples": ["safety_monitoring", "emergency_response"]
        },
        "high": {
            "range": [70, 89],
            "description": "Primary activity functions",
            "deferrable": true,
            "max_deferral": 60,
            "examples": ["toy_recognition", "emotion_detection"]
        },
        "medium": {
            "range": [40, 69],
            "description": "Supporting functions",
            "deferrable": true,
            "max_deferral": 300,
            "examples": ["conversation_analysis", "progress_tracking"]
        },
        "low": {
            "range": [10, 39],
            "description": "Background analysis",
            "deferrable": true,
            "max_deferral": 600,
            "examples": ["behavioral_pattern_analysis", "system_optimization"]
        },
        "reflection": {
            "range": [1, 9],
            "description": "Deep analysis tasks",

```

```

    "deferrable": true,
    "max_deferral": 1800,
    "examples": ["long_term_trend_analysis", "model_retraining"]
  },
  "context_boosts": {
    "tea_party_active": {
      "toy_recognition": 20,
      "child_emotion_detection": 25,
      "ar_overlay_render": 15
    },
    "creative_session": {
      "drawing_canvas_render": 20,
      "color_palette_management": 15,
      "ai_assistance": 10
    },
    "crisis_emergency": {
      "safety_monitoring": 30,
      "emergency_response": 25,
      "alert_generation": 20
    }
  }
}

```

6. Benefits & Implementation Phases

A. Key Benefits:

1. **Adaptive Performance:** System maintains responsiveness during critical activities
2. **Resource Efficiency:** Non-essential tasks deferred to idle periods
3. **Deep Learning:** Reflection periods enable sophisticated analysis without impacting user experience
4. **Thermal Management:** Prevents overheating by reducing load during high-temperature periods
5. **Predictive Optimization:** Learns user patterns to pre-allocate resources

B. Implementation Phases:

Phase 1: Foundation (2 weeks)

- Basic task prioritization system
- Simple deferral mechanism
- Context detection foundation

Phase 2: Advanced Scheduling (3 weeks)

- Three-layer execution system
- Intelligent deferral with triggers
- Resource monitoring integration

Phase 3: Reflection Engine (2 weeks)

- Reflection period management

- Analysis modules for deferred data
- Insight generation system

Phase 4: Optimization & Integration (3 weeks)

- Integration with all existing modules
- Performance optimization for RPi5
- Real-world testing and refinement

Conclusion

The **Adaptive Resource Management Engine (ARME)** transforms resource constraints into **strategic opportunities** by:

1. **Context-Aware Prioritization:** Essential functions get resources when needed
2. **Intelligent Deferral:** Non-essential analysis saved for idle periods
3. **Deep Reflection:** System learns and optimizes during downtime
4. **Predictive Allocation:** Anticipates needs based on patterns

This creates a system that feels **more responsive** to users while actually doing **more sophisticated processing** in the background during reflection periods. The tea party can run smoothly with all interactive elements, while behavioral analysis and system optimization happen later when the child is resting.

Key Innovation: The system learns to **predict its own idle periods** and schedules deep analysis accordingly, creating a virtuous cycle where better understanding leads to better resource prediction, which leads to more efficient analysis.

This approach embodies the **Peter Protocol's efficiency principle**: "Protect without burdening, understand without intruding, learn without demanding."